

MySQL Stored Procedures & Triggers - Complete Training Script (Single Panel)

This script is designed to teach trainees:

- ✓ DDL
 - ✓ DML
 - ✓ Stored Procedures
 - ✓ IN / OUT / INOUT Parameters
 - ✓ Variables
 - ✓ IF / CASE
 - ✓ Loops
 - ✓ Cursors
 - ✓ Error Handling
 - ✓ Dynamic SQL
 - ✓ BEFORE Triggers
 - ✓ AFTER Triggers
 - ✓ Audit Logging
 - ✓ Salary History Tracking
 - ✓ Business Rule Validation
-

1. CREATE TABLES

```
DROP TABLE IF EXISTS salary_history;  
DROP TABLE IF EXISTS employee_audit;  
DROP TABLE IF EXISTS employees;  
DROP TABLE IF EXISTS departments;
```

```
CREATE TABLE departments  
(  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(100),  
    budget DECIMAL(15,2)  
);
```

```
CREATE TABLE employees  
(  
    employee_id INT PRIMARY KEY,
```

```
employee_name VARCHAR(100),
department_id INT,
salary DECIMAL(10,2),
hire_date DATE,
status VARCHAR(20),

FOREIGN KEY (department_id)
REFERENCES departments(department_id)
);

CREATE TABLE employee_audit
(
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT,
    action_type VARCHAR(20),
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE salary_history
(
    history_id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT,
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    changed_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

2. INSERT SAMPLE DATA

INSERT INTO departments VALUES

```
(1,'IT',1000000),
(2,'HR',500000),
(3,'Finance',750000),
(4,'Sales',900000);
```

INSERT INTO employees VALUES

```
(101,'Arun Kumar',1,70000,'2022-01-10','ACTIVE'),
(102,'Priya Sharma',1,85000,'2021-05-15','ACTIVE'),
(103,'Rahul Verma',2,50000,'2023-02-20','ACTIVE'),
(104,'Sneha Reddy',3,95000,'2020-08-12','ACTIVE'),
```

```
(105,'Vikram Singh',4,60000,'2022-10-01','ACTIVE');
```

STORED PROCEDURES

3. PROCEDURE WITHOUT PARAMETERS

```
DELIMITER $$
```

```
CREATE PROCEDURE sp_get_all_employees()  
BEGIN
```

```
    SELECT *  
    FROM employees;
```

```
END $$
```

```
DELIMITER ;
```

Execute

```
CALL sp_get_all_employees();
```

4. PROCEDURE WITH IN PARAMETER

```
DELIMITER $$
```

```
CREATE PROCEDURE sp_get_employee_by_id  
(  
    IN p_employee_id INT  
)  
BEGIN
```

```
    SELECT *  
    FROM employees  
    WHERE employee_id = p_employee_id;
```

```
END $$
```

DELIMITER ;

Execute

CALL sp_get_employee_by_id(101);

5. PROCEDURE WITH OUT PARAMETER

DELIMITER \$\$

CREATE PROCEDURE sp_employee_count

(
 OUT total_employees INT
)

BEGIN

SELECT COUNT(*)
 INTO total_employees
 FROM employees;

END \$\$

DELIMITER ;

Execute

CALL sp_employee_count(@cnt);

SELECT @cnt;

6. PROCEDURE WITH INOUT PARAMETER

DELIMITER \$\$

CREATE PROCEDURE sp_add_bonus

(
 INOUT p_salary DECIMAL(10,2)
)

BEGIN

```
    SET p_salary = p_salary + 5000;

END $$

DELIMITER ;

Execute

SET @salary = 60000;

CALL sp_add_bonus(@salary);

SELECT @salary;
```

7. IF ELSE PROCEDURE

```
DELIMITER $$

CREATE PROCEDURE sp_salary_grade
(
    IN p_salary DECIMAL(10,2)
)
BEGIN

    IF p_salary >= 90000 THEN

        SELECT 'GRADE A' AS Grade;

    ELSEIF p_salary >= 70000 THEN

        SELECT 'GRADE B' AS Grade;

    ELSE

        SELECT 'GRADE C' AS Grade;

    END IF;

END $$

DELIMITER ;

Execute
```

```
CALL sp_salary_grade(85000);
```

8. CASE PROCEDURE

```
DELIMITER $$
```

```
CREATE PROCEDURE sp_salary_grade_case  
(  
    IN p_salary DECIMAL(10,2)  
)  
BEGIN
```

```
    SELECT CASE
```

```
        WHEN p_salary >= 90000 THEN 'A'
```

```
        WHEN p_salary >= 70000 THEN 'B'
```

```
        ELSE 'C'
```

```
    END AS Grade;
```

```
END $$
```

```
DELIMITER ;
```

9. WHILE LOOP PROCEDURE

```
DELIMITER $$
```

```
CREATE PROCEDURE sp_loop_demo()  
BEGIN
```

```
    DECLARE counter INT DEFAULT 1;
```

```
    WHILE counter <= 5 DO
```

```
        SELECT counter;
```

```
        SET counter = counter + 1;
```

```
END WHILE;

END $$

DELIMITER ;

Execute

CALL sp_loop_demo();
```

10. REPEAT LOOP PROCEDURE

```
DELIMITER $$

CREATE PROCEDURE sp_repeat_demo()
BEGIN

    DECLARE counter INT DEFAULT 1;

    REPEAT

        SELECT counter;

        SET counter = counter + 1;

    UNTIL counter > 5

    END REPEAT;

END $$

DELIMITER ;
```

11. LOOP WITH LEAVE

```
DELIMITER $$

CREATE PROCEDURE sp_leave_demo()
BEGIN
```

```
DECLARE counter INT DEFAULT 1;

my_loop: LOOP

    IF counter > 5 THEN
        LEAVE my_loop;
    END IF;

    SELECT counter;

    SET counter = counter + 1;

END LOOP;

END $$

DELIMITER ;
```

12. CURSOR EXAMPLE

```
DELIMITER $$

CREATE PROCEDURE sp_cursor_demo()
BEGIN

    DECLARE done INT DEFAULT FALSE;

    DECLARE v_name VARCHAR(100);

    DECLARE emp_cursor CURSOR FOR

        SELECT employee_name
        FROM employees;

    DECLARE CONTINUE HANDLER
    FOR NOT FOUND
    SET done = TRUE;

    OPEN emp_cursor;

    read_loop: LOOP
```



```
    FETCH emp_cursor
    INTO v_name;

    IF done THEN
        LEAVE read_loop;
    END IF;

    SELECT v_name;

END LOOP;

CLOSE emp_cursor;

END $$

DELIMITER ;
```

13. EXCEPTION HANDLING

```
DELIMITER $$

CREATE PROCEDURE sp_error_handling()
BEGIN

    DECLARE EXIT HANDLER
    FOR SQLEXCEPTION

    BEGIN

        SELECT 'ERROR OCCURRED';

    END;

    INSERT INTO employees
    VALUES
    (101,'Duplicate Employee',1,50000,CURDATE(),'ACTIVE');

END $$

DELIMITER ;
```

14. DYNAMIC SQL

DELIMITER \$\$

```
CREATE PROCEDURE sp_dynamic_sql
(
    IN p_table_name VARCHAR(50)
)
BEGIN

    SET @sql =
    CONCAT('SELECT * FROM ', p_table_name);

    PREPARE stmt FROM @sql;

    EXECUTE stmt;

    DEALLOCATE PREPARE stmt;

END $$

DELIMITER ;

Execute

CALL sp_dynamic_sql('employees');
```

TRIGGERS

15. BEFORE INSERT TRIGGER

Salary cannot be negative

DELIMITER \$\$

CREATE TRIGGER trg_before_insert_salary

BEFORE INSERT

```
ON employees

FOR EACH ROW

BEGIN

    IF NEW.salary < 0 THEN

        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Salary cannot be negative';

    END IF;

END $$

DELIMITER ;
```

16. BEFORE INSERT TRIGGER

```
Default Status

DELIMITER $$

CREATE TRIGGER trg_default_status

BEFORE INSERT

ON employees

FOR EACH ROW

BEGIN

    IF NEW.status IS NULL THEN

        SET NEW.status='ACTIVE';

    END IF;

END $$
```

DELIMITER ;

17. BEFORE UPDATE TRIGGER

Prevent Salary Reduction

DELIMITER \$\$

CREATE TRIGGER trg_salary_validation

BEFORE UPDATE

ON employees

FOR EACH ROW

BEGIN

IF NEW.salary < OLD.salary THEN

SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT =
'Salary reduction not allowed';

END IF;

END \$\$

DELIMITER ;

18. AFTER INSERT TRIGGER

Audit Insert

DELIMITER \$\$

CREATE TRIGGER trg_after_insert

```
AFTER INSERT

ON employees

FOR EACH ROW

BEGIN

    INSERT INTO employee_audit
    (
        employee_id,
        action_type,
        new_salary
    )
    VALUES
    (
        NEW.employee_id,
        'INSERT',
        NEW.salary
    );

END $$

DELIMITER ;
```

19. AFTER UPDATE TRIGGER

Audit Update

```
DELIMITER $$

CREATE TRIGGER trg_after_update

AFTER UPDATE

ON employees

FOR EACH ROW

BEGIN

    INSERT INTO employee_audit
```

```
(
    employee_id,
    action_type,
    old_salary,
    new_salary
)
VALUES
(
    NEW.employee_id,
    'UPDATE',
    OLD.salary,
    NEW.salary
);

END $$
```

```
DELIMITER ;
```

20. AFTER UPDATE TRIGGER

Salary History

```
DELIMITER $$
```

```
CREATE TRIGGER trg_salary_history
```

```
AFTER UPDATE
```

```
ON employees
```

```
FOR EACH ROW
```

```
BEGIN
```

```
IF OLD.salary <> NEW.salary THEN
```

```
    INSERT INTO salary_history
```

```
    (
        employee_id,
        old_salary,
        new_salary
    )
```

```
VALUES
(
    NEW.employee_id,
    OLD.salary,
    NEW.salary
);

END IF;

END $$

DELIMITER ;
```

21. AFTER DELETE TRIGGER

Audit Delete

```
DELIMITER $$

CREATE TRIGGER trg_after_delete

AFTER DELETE

ON employees

FOR EACH ROW

BEGIN

    INSERT INTO employee_audit
    (
        employee_id,
        action_type,
        old_salary
    )
    VALUES
    (
        OLD.employee_id,
        'DELETE',
        OLD.salary
    );
```

END \$\$

DELIMITER ;

22. TEST ALL TRIGGERS

INSERT

```
INSERT INTO employees
VALUES
(
106,
'Anjali Gupta',
2,
65000,
CURDATE(),
'ACTIVE'
);
```

UPDATE

```
UPDATE employees
SET salary = 75000
WHERE employee_id = 106;
```

DELETE

```
DELETE
FROM employees
WHERE employee_id = 106;
```

23. VERIFY RESULTS

```
SELECT *
FROM employee_audit
ORDER BY action_time DESC;
SELECT *
FROM salary_history
ORDER BY changed_on DESC;
```

Frequently Asked Interview Scenarios

Scenario	Solution
Employee Audit	AFTER INSERT/UPDATE/DELETE Trigger
Salary Validation	BEFORE INSERT Trigger
Prevent Salary Reduction	BEFORE UPDATE Trigger
Auto Default Values	BEFORE INSERT Trigger
Generate Reports	Stored Procedure
Batch Processing	Stored Procedure
Dynamic Queries	Dynamic SQL
Exception Handling	EXIT HANDLER
Row-by-Row Processing	Cursor
Iteration	WHILE / REPEAT / LOOP
Return Values	OUT Parameter
Modify Input Value	INOUT Parameter
Business Rules	Triggers
Maintain History Table	AFTER UPDATE Trigger

Trigger Execution Order

INSERT
↓
BEFORE INSERT
↓
INSERT OCCURS
↓
AFTER INSERT
UPDATE
↓
BEFORE UPDATE

↓
UPDATE OCCURS
↓
AFTER UPDATE
DELETE
↓
BEFORE DELETE
↓
DELETE OCCURS
↓
AFTER DELETE

This covers nearly all MySQL stored procedure and trigger concepts a fresher, intermediate developer, Data Engineer, ETL developer, or interview candidate should know.